

Trabalho Final de Segurança Computacional

Implementação Didática do Protocolo Kerberos

Disciplina: Segurança Computacional

Instituição: Universidade de Brasília

Alunos:

Henrique Luz Martins - 200061691

Gustavo Choueiri - 232014010

1. Introdução

Este trabalho apresenta uma implementação didática do protocolo Kerberos, com foco no estudo prático de autenticação baseada em criptografia de chave simétrica, emissão de tickets, derivação de chaves a partir de senha e autenticação mútua entre cliente e serviço.

A implementação foi desenvolvida em Python e segue a ideia central do fluxo Kerberos apresentado em aula: o cliente primeiro se autentica junto ao Servidor de Autenticação, depois solicita ao Ticket Granting Server um ticket para acessar um serviço específico e, por fim, utiliza esse ticket para se autenticar perante o serviço protegido.

O serviço protegido escolhido foi um sistema simples de notas. A complexidade do serviço em si é propositalmente baixa, pois o objetivo principal do trabalho é demonstrar corretamente o funcionamento do protocolo Kerberos, e não desenvolver uma aplicação final complexa.

A solução utiliza exclusivamente criptografia de chave simétrica para proteger tickets, autenticadores e mensagens sensíveis entre as partes. Não foram utilizadas bibliotecas prontas de Kerberos. A biblioteca `cryptography` foi usada apenas para primitivas criptográficas básicas, como PBKDF2-HMAC-SHA256 e AES-GCM.

2. Arquitetura da solução

A arquitetura desenvolvida é composta por quatro componentes principais:

1. Cliente Kerberos (`client.py`)

Responsável por receber o usuário e a senha, derivar a chave do usuário, solicitar tickets ao AS e ao TGS e acessar o serviço protegido de notas.

2. Servidor de Autenticação - AS (`as_server.py`)

Responsável pela primeira etapa de autenticação. O AS valida se o usuário existe, gera uma chave de sessão entre cliente e TGS e emite o Ticket Granting Ticket, também chamado de TGT.

3. Ticket Granting Server - TGS (`tgs_server.py`)

Responsável por receber o TGT emitido pelo AS, validar o autenticador enviado pelo cliente e emitir um ticket específico para o serviço protegido.

4. Serviço protegido de notas (`notes_service.py`)

Responsável por validar o ticket de serviço e o autenticador do cliente. Após a validação, permite que o usuário adicione e liste notas. O serviço também participa da autenticação mútua, respondendo ao cliente de forma criptografada.

Além desses arquivos, há dois módulos auxiliares:

- `crypto_utils.py`: contém funções de criptografia, derivação de chave, serialização JSON, envio e recebimento de mensagens via socket.
- `kerberos_config.py`: contém configurações de portas, identificadores dos serviços, usuários de teste e chaves simétricas internas.

3. Fluxo geral do protocolo implementado

O fluxo implementado segue três grandes etapas.

Na primeira etapa, o cliente envia uma requisição ao AS informando sua identidade e solicitando acesso ao TGS. O AS gera uma chave de sessão entre cliente e TGS. Em seguida, o AS cria o TGT, criptografado com a chave secreta compartilhada entre AS e TGS. O AS também envia ao cliente uma resposta criptografada com a chave derivada da senha do usuário.

Na segunda etapa, o cliente usa o TGT para solicitar ao TGS um ticket para o serviço de notas. Para provar que é o mesmo cliente para quem o TGT foi emitido, ele também envia um autenticador criptografado com a chave de sessão Cliente-TGS. O TGS valida o TGT, valida o autenticador e gera uma nova chave de sessão, agora entre cliente e serviço. Em seguida, emite um ticket de serviço criptografado com a chave secreta do serviço de notas.

Na terceira etapa, o cliente envia ao serviço de notas o ticket de serviço e um novo autenticador. O serviço descriptografa o ticket, obtém a chave de sessão Cliente-Serviço, valida o autenticador e processa a operação solicitada. Para confirmar a autenticação mútua, o serviço responde ao cliente com uma mensagem criptografada contendo o valor `timestamp + 1`. O cliente valida esse valor e confirma que o serviço realmente possui a chave de sessão correta.

Dessa forma, o cliente não envia sua senha pela rede, o serviço não precisa conhecer a senha do usuário e o acesso ao serviço protegido depende da posse de tickets válidos e das chaves de sessão corretas.

4. Implementação do Servidor de Autenticação (AS)

O Servidor de Autenticação foi implementado no arquivo `as_server.py`. Ele representa a primeira etapa do fluxo Kerberos.

O cliente inicia o processo enviando uma mensagem do tipo `AS_REQ`, contendo seu identificador de usuário e o identificador do TGS que deseja acessar. Nesta implementação, os usuários cadastrados ficam definidos em `kerberos_config.py`, com os usuários de teste `alice` e `bob`.

Ao receber uma requisição, o AS verifica se o usuário existe. Caso o usuário não esteja cadastrado, o servidor retorna uma mensagem de erro. Caso o usuário exista, o AS recupera a chave simétrica associada ao usuário.

Essa chave não é armazenada como senha em texto puro: ela é derivada a partir da senha por meio da função `derive_client_key`, definida em `crypto_utils.py`.

Depois da validação do usuário, o AS gera uma chave de sessão aleatória entre cliente e TGS, chamada nesta implementação de `client_tgs_session_key`. Essa chave será usada posteriormente para proteger a comunicação entre o cliente e o TGS.

O AS então cria o TGT, ou Ticket Granting Ticket. Esse ticket contém:

- tipo do ticket;
- identificador do cliente;
- identificador do TGS;
- chave de sessão Cliente-TGS;
- horário de emissão;
- horário de expiração.

O TGT é criptografado com a chave secreta compartilhada entre AS e TGS, chamada no código de `KDC_TGS_KEY`. Isso significa que o cliente recebe o TGT, mas não consegue ler seu conteúdo interno. Ele apenas armazena e repassa esse ticket ao TGS na etapa seguinte.

Além do TGT, o AS envia ao cliente uma resposta criptografada com a chave do próprio usuário. Essa resposta contém a chave de sessão Cliente-TGS e o TGT. Como essa mensagem é criptografada com a chave derivada da senha do usuário, apenas um cliente que digitou a senha correta consegue descriptografar a resposta do AS.

Esse comportamento foi testado na execução prática. Com a senha correta de `alice`, o cliente conseguiu descriptografar a resposta do AS e obter o TGT. Com uma senha errada, a descriptografia falhou, mostrando que a autenticação realmente depende da chave derivada da senha.

5. Implementação do Ticket Granting Server (TGS)

O Ticket Granting Server foi implementado no arquivo `tgs_server.py`. Ele representa a segunda etapa do protocolo Kerberos.

Depois de obter o TGT junto ao AS, o cliente envia ao TGS uma mensagem do tipo `TGS_REQ`. Essa mensagem contém:

- o identificador do serviço solicitado;
- o TGT recebido do AS;
- um autenticador criptografado com a chave de sessão Cliente-TGS.

O TGS primeiro descriptografa o TGT usando a chave secreta compartilhada com o AS, chamada `KDC_TGS_KEY`. Se a descriptografia falhar, o ticket é rejeitado. Se funcionar, o TGS verifica se o ticket realmente é um TGT, se foi emitido para aquele TGS e se ainda está dentro do prazo de validade.

Depois disso, o TGS obtém do TGT a chave de sessão Cliente-TGS. Com essa chave, ele descriptografa o autenticador enviado pelo cliente. O autenticador contém o identificador do cliente e um timestamp.

Essa etapa é importante porque impede que alguém simplesmente copie um TGT e tente usá-lo sem possuir a chave de sessão correta. O cliente precisa provar que conhece a chave de sessão Cliente-TGS, e faz isso

criando um autenticador válido.

Após validar o TGT e o autenticador, o TGS gera uma nova chave de sessão, agora entre cliente e serviço, chamada `client_service_session_key`.

Em seguida, o TGS emite o ticket de serviço. Esse ticket contém:

- tipo do ticket;
- identificador do cliente;
- identificador do serviço;
- chave de sessão Cliente-Serviço;
- horário de emissão;
- horário de expiração.

O ticket de serviço é criptografado com a chave secreta do serviço de notas, chamada `SERVICE_KEY`. Dessa forma, o cliente recebe o ticket, mas não consegue alterar seu conteúdo sem invalidar a criptografia. Apenas o serviço protegido consegue descriptografá-lo.

Por fim, o TGS envia ao cliente uma resposta criptografada com a chave de sessão Cliente-TGS. Essa resposta contém a chave de sessão Cliente-Serviço e o ticket de serviço. Assim, o cliente passa a ter os dados necessários para se autenticar perante o serviço protegido.

6. Derivação de chaves a partir da senha do usuário

A autenticação do usuário começa com uma senha. Porém, a senha não é usada diretamente como chave criptográfica. Em vez disso, o cliente deriva uma chave simétrica a partir da senha informada.

A função usada para isso está em `crypto_utils.py`, com o nome `derive_client_key`. A função de derivação adotada foi **PBKDF2-HMAC-SHA256**.

A derivação utiliza:

- a senha digitada pelo usuário;
- um salt baseado no domínio da aplicação e no nome do usuário;
- 200.000 iterações;
- saída de 32 bytes, adequada para uso com AES-256.

O salt usado segue a ideia de associar a chave ao contexto do usuário. Nesta implementação, ele é formado por:

```
KERBEROS-DEMO.UNB:nome_do_usuario
```

Por exemplo, para a usuária `alice`, o salt fica associado a `KERBEROS-DEMO.UNB:alice`.

O uso de PBKDF2 é importante porque torna ataques de força bruta mais custosos do que simplesmente aplicar uma função hash uma única vez. Como a função realiza muitas iterações, cada tentativa de senha fica computacionalmente mais cara para um atacante.

Na prática, quando o usuário digita a senha correta, o cliente consegue derivar a mesma chave esperada pelo AS e descriptografar a resposta **AS_REP**. Quando a senha está errada, a chave derivada também fica errada e a descriptografia falha.

7. Obtenção e uso dos tickets

A implementação utiliza dois tipos principais de tickets.

O primeiro é o **TGT**, ou Ticket Granting Ticket. Ele é emitido pelo AS e serve para permitir que o cliente solicite tickets de serviço ao TGS sem precisar reenviar a senha. O TGT é criptografado com a chave compartilhada entre AS e TGS, então o cliente não consegue ler nem alterar o conteúdo interno do ticket.

O segundo é o **ticket de serviço**. Ele é emitido pelo TGS e permite que o cliente acesse o serviço protegido de notas. Esse ticket é criptografado com a chave secreta do serviço, então apenas o serviço de notas consegue descriptografá-lo.

Cada ticket possui tempo de emissão e tempo de expiração. Isso limita o tempo de uso de um ticket e reduz o risco de reutilização indevida. Nesta implementação, o tempo de validade é definido em **kerberos_config.py** pela constante **TICKET_LIFETIME_SECONDS**.

O uso dos tickets ocorre da seguinte forma:

1. O cliente recebe o TGT do AS.
2. O cliente envia o TGT ao TGS.
3. O TGS valida o TGT e emite o ticket de serviço.
4. O cliente envia o ticket de serviço ao serviço de notas.
5. O serviço valida o ticket e permite a execução da operação solicitada.

Esse processo evita que a senha do usuário seja enviada ao serviço protegido. O serviço confia no ticket emitido pelo TGS, e não precisa conhecer a senha do usuário.

8. Autenticadores e prevenção de replay

Além dos tickets, a implementação utiliza autenticadores.

O ticket sozinho não é suficiente, porque alguém poderia capturar um ticket válido e tentar reutilizá-lo. Para reduzir esse risco, o cliente também envia um autenticador criptografado com a chave de sessão correspondente.

Na comunicação com o TGS, o autenticador é criptografado com a chave de sessão Cliente-TGS. Na comunicação com o serviço de notas, o autenticador é criptografado com a chave de sessão Cliente-Serviço.

Cada autenticador contém:

- identificador do cliente;
- timestamp atual.

O servidor que recebe o autenticador verifica duas coisas principais:

1. se o cliente indicado no autenticador é o mesmo cliente indicado no ticket;

2. se o timestamp está dentro de uma janela de tempo aceitável.

Nesta implementação, a janela de tempo aceitável é definida por `MAX_CLOCK_SKEW_SECONDS`, no arquivo `crypto_utils.py`. O valor usado foi de 120 segundos.

Com isso, a implementação demonstra a ideia central do Kerberos: não basta possuir um ticket; é necessário também provar que se conhece a chave de sessão associada a ele.

9. Autenticação mútua

A autenticação mútua foi implementada na comunicação entre o cliente e o serviço protegido de notas.

Depois que o cliente envia o ticket de serviço e o autenticador, o serviço valida ambos. Se tudo estiver correto, o serviço responde ao cliente com uma mensagem criptografada usando a chave de sessão Cliente-Serviço.

Essa resposta contém o valor:

```
timestamp + 1
```

O cliente havia enviado o timestamp original no autenticador. Ao receber a resposta, ele descriptografa a mensagem usando a chave de sessão Cliente-Serviço e verifica se o valor recebido corresponde ao timestamp enviado mais um.

Se a verificação for bem-sucedida, o cliente imprime:

```
[CLIENT] Autenticacao mutua confirmada.
```

Isso demonstra que o serviço realmente conseguiu descriptografar o ticket de serviço, obteve a chave de sessão correta e respondeu usando essa mesma chave. Portanto, o cliente também autentica o serviço.

10. Algoritmos criptográficos utilizados

A implementação utiliza dois mecanismos criptográficos principais: PBKDF2-HMAC-SHA256 para derivação de chaves e AES-GCM para criptografia simétrica autenticada.

A função **PBKDF2-HMAC-SHA256** foi usada para derivar chaves a partir das senhas dos usuários. Essa escolha é adequada porque PBKDF2 é uma função de derivação de chaves projetada para tornar ataques de força bruta mais custosos. O uso de HMAC-SHA256 fornece uma base criptográfica forte, e o número de iterações aumenta o custo computacional de cada tentativa de senha.

A criptografia simétrica foi implementada com **AES-GCM**. O AES é um algoritmo de cifra simétrica amplamente utilizado. O modo GCM foi escolhido porque fornece confidencialidade e autenticação dos dados criptografados. Isso significa que, além de esconder o conteúdo da mensagem, ele também permite detectar alterações indevidas no texto cifrado.

As chaves de sessão são geradas com bytes aleatórios por meio de `os.urandom`, evitando que sejam previsíveis. Essas chaves são usadas somente durante o tempo de validade dos tickets.

A implementação não utiliza RSA, certificados digitais, TLS, mTLS nem bibliotecas prontas de Kerberos. Isso foi feito para respeitar a restrição do trabalho, que exige a implementação do protocolo com base em criptografia de chave simétrica e primitivas criptográficas básicas.

11. Serviço protegido implementado

O serviço protegido escolhido foi um sistema simples de notas, implementado no arquivo `notes_service.py`.

Após a autenticação Kerberos, o usuário pode realizar duas operações:

- adicionar uma nota;
- listar as notas já adicionadas.

O serviço de notas só processa uma requisição depois de validar o ticket de serviço e o autenticador enviado pelo cliente. Caso o ticket esteja inválido, expirado, criptografado com a chave errada ou tenha sido emitido para outro serviço, a requisição é rejeitada.

O objetivo desse serviço não é ser uma aplicação complexa, mas sim demonstrar que um recurso protegido só pode ser acessado após a autenticação Kerberos. Dessa forma, o serviço cumpre o requisito de existir pelo menos uma aplicação protegida pelo protocolo.

12. Testes realizados

Foram realizados testes manuais pelo terminal do VSCode.

No primeiro teste, os três servidores foram iniciados separadamente:

```
py as_server.py
py tgs_server.py
py notes_service.py
```

Depois, o cliente foi executado com:

```
py client.py
```

Usando o usuário `alice` e a senha correta `alice123`, o cliente conseguiu obter o TGT junto ao AS, obter o ticket de serviço junto ao TGS e acessar o serviço protegido de notas.

Durante a execução, o cliente exibiu mensagens indicando:

```
AS_REP recebida e descriptografada com sucesso.  
TGT obtido.  
TGS_REP recebida e descriptografada com sucesso.  
Ticket de servico obtido.  
Autenticacao mutua confirmada.
```

Também foi testada a adição de uma nota e a listagem das notas cadastradas. O serviço respondeu corretamente, mostrando que a requisição foi aceita após a validação Kerberos.

Em outro teste, foi usada uma senha incorreta para a usuária **alice**. Nesse caso, o cliente não conseguiu descriptografar a resposta do AS e exibiu erro de autenticação. Esse teste confirmou que a chave do usuário realmente depende da senha informada.

13. Principais dificuldades encontradas

Uma das principais dificuldades foi separar corretamente as responsabilidades de cada componente do Kerberos. O AS não deve atuar como serviço final, o TGS não deve autenticar diretamente a senha do usuário e o serviço protegido não deve conhecer a senha do cliente.

Outra dificuldade foi entender a diferença entre ticket e autenticador. O ticket é emitido por um servidor confiável e criptografado para outro servidor. Já o autenticador é criado pelo cliente e serve para provar que ele conhece a chave de sessão associada ao ticket.

Também foi necessário tomar cuidado para que o cliente não tivesse acesso indevido ao conteúdo interno dos tickets. Por isso, o TGT foi criptografado com a chave compartilhada entre AS e TGS, e o ticket de serviço foi criptografado com a chave secreta do serviço.

Por fim, houve dificuldade em organizar a execução prática, porque a aplicação depende de quatro processos rodando ao mesmo tempo: AS, TGS, serviço de notas e cliente.

14. Aprendizados obtidos

O trabalho ajudou a compreender que o Kerberos não é apenas um mecanismo de login, mas um protocolo de distribuição e validação de tickets baseado em confiança entre entidades.

Também ficou claro que a senha do usuário não precisa ser enviada pela rede. Ela é usada localmente para derivar uma chave, e essa chave permite descriptografar a resposta do AS.

Outro aprendizado importante foi o papel das chaves de sessão. Elas evitam que uma mesma chave permanente seja usada para proteger toda a comunicação. Cada etapa gera uma chave temporária para um relacionamento específico: Cliente-TGS ou Cliente-Serviço.

A autenticação mútua também ficou mais clara na prática. O cliente não apenas prova sua identidade ao serviço, mas também verifica se o serviço realmente possui a chave correta, confirmando que está falando com a entidade esperada.

15. Conclusão

A implementação desenvolvida cumpriu o objetivo de demonstrar, de forma prática, o funcionamento básico do protocolo Kerberos.

Foram implementados um Servidor de Autenticação, um Ticket Granting Server, um serviço protegido de notas e um cliente capaz de executar o fluxo completo de autenticação. A solução usa senha de usuário, KDF, tickets, autenticadores, chaves de sessão, criptografia simétrica e autenticação mútua.

Embora seja uma versão didática e simplificada, a arquitetura representa os principais conceitos estudados em aula: autenticação centralizada, emissão de tickets, separação entre AS e TGS, uso de chaves simétricas e validação do cliente perante o serviço protegido.

Assim, o trabalho permite observar na prática como o Kerberos reduz a necessidade de reenviar senhas, distribui chaves de sessão e permite que serviços protegidos autenticuem clientes de forma segura.